

2D sparse Table

```

const int LG = 11;

template<typename T, class CMP = function<T(const T &, const T &>>
struct SP2d {
    CMP func;
    int m, n;
    vector<vector<int>> sp[LG][LG];

    SP2d(const vector<vector<T>> &arr, const CMP &f) : func(f) {
        n = arr.size(), m = arr[0].size();
        for (int i = 0; i < LG; ++i) {
            for (int j = 0; j < LG; ++j) {
                sp[i][j].assign(n, vector<int>(m));
            }
        }
        for (int ir = 0; ir < n; ++ir) {
            for (int ic = 0; ic < m; ++ic)
                sp[0][0][ir][ic] = arr[ir][ic];
        }
        for (int a = 0; a < LG; a++) {
            for (int b = 0; b < LG; b++) {
                if (a + b == 0) continue;
                for (int i = 0; i + (1 << a) <= n; i++) {
                    for (int j = 0; j + (1 << b) <= m; j++) {
                        if (!a) {
                            sp[a][b][i][j] = func(
                                sp[a][b - 1][i][j],
                                sp[a][b - 1][i][j + (1 << (b - 1))]
                            );
                        } else {
                            sp[a][b][i][j] = func(
                                sp[a - 1][b][i][j],
                                sp[a - 1][b][i + (1 << (a - 1))][j]
                            );
                        }
                    }
                }
            }
        }
    }

    T query(int x1, int y1, int x2, int y2) {
        int lgX = __lg(x2 - x1 + 1);
        int lgY = __lg(y2 - y1 + 1);
        T val1 = func(
            sp[lgX][lgY][x1][y1],
            sp[lgX][lgY][x1][y2 - (1 << lgY) + 1]
        );
    }
};

```

```

    T val2 = func(
        sp[lgX][lgY][x2 - (1 << lgX) + 1][y1],
        sp[lgX][lgY][x2 - (1 << lgX) + 1][y2 - (1 << lgY) + 1]
    );
    return func(val1, val2);
}
};

```

2D BIT Online

```

#define vi vector<ll>
#define vvi vector<vi>
#define vvvi vector<vvi>

struct BIT2D {
private:
    vvvi M, A;

    void upd2(vvvi &t, int x, int y, ll mul, ll add) {
        for (int i = x; i < t.size(); i += i & -i) {
            for (int j = y; j < t[0].size(); j += j & -j) {
                t[i][j][0] += mul;
                t[i][j][1] += add;
            }
        }
    }

    void upd1(int x, int y1, int y2, ll mul, ll add) {
        upd2(M, x, y1, mul, -mul * (y1 - 1));
        upd2(M, x, y2, -mul, mul * y2);
        upd2(A, x, y1, add, -add * (y1 - 1));
        upd2(A, x, y2, -add, add * y2);
    }

    ll query2(vvvi &t, int x, int y) {
        ll mul = 0, add = 0;
        for (int i = y; i > 0; i -= i & -i) {
            mul += t[x][i][0];
            add += t[x][i][1];
        }
        return mul * y + add;
    }

    ll query1(int x, int y) {
        ll mul = 0, add = 0;
        for (int i = x; i > 0; i -= i & -i) {
            mul += query2(M, i, y);
            add += query2(A, i, y);
        }
    }
}

```

```

    }
    return mul * x + add;
}

public:
BIT2D(int n, int m) {
    M.assign(n + 1, vvi(m + 1, vi(2, 0)));
    A = M;
}

ll query(int x1, int y1, int x2, int y2) {
    return query1(x2, y2) -
           query1(x1 - 1, y2) -
           query1(x2, y1 - 1) +
           query1(x1 - 1, y1 - 1);
}

void update(int x1, int y1, int x2, int y2, ll val) {
    upd1(x1, y1, y2, val, -val * (x1 - 1));
    upd1(x2, y1, y2, -val, val * x2);
}

};

```

2D BIT Point Update Range Query :

```

#define vi vector<ll>
#define vvi vector<vi>
#define vvvi vector<vvi>

struct BIT2D {
private:
    vvvi M, A;

    void upd2(vvvi &t, int x, int y, ll mul, ll add) {
        for (int i = x; i < t.size(); i += i & -i) {
            for (int j = y; j < t[0].size(); j += j & -j) {
                t[i][j][0] += mul;
                t[i][j][1] += add;
            }
        }
    }

    void upd1(int x, int y1, int y2, ll mul, ll add) {
        upd2(M, x, y1, mul, -mul * (y1 - 1));
        upd2(M, x, y2, -mul, mul * y2);
        upd2(A, x, y1, add, -add * (y1 - 1));
        upd2(A, x, y2, -add, add * y2);
    }
}

```

```

ll query2(vvvi &t, int x, int y) {
    ll mul = 0, add = 0;
    for (int i = y; i > 0; i -= i & -i) {
        mul += t[x][i][0];
        add += t[x][i][1];
    }
    return mul * y + add;
}

ll query1(int x, int y) {
    ll mul = 0, add = 0;
    for (int i = x; i > 0; i -= i & -i) {
        mul += query2(M, i, y);
        add += query2(A, i, y);
    }
    return mul * x + add;
}

public:
BIT2D(int n, int m) {
    M.assign(n + 1, vvi(m + 1, vi(2, 0)));
    A = M;
}

ll query(int x1, int y1, int x2, int y2) {
    return query1(x2, y2) -
        query1(x1 - 1, y2) -
        query1(x2, y1 - 1) +
        query1(x1 - 1, y1 - 1);
}

void update(int x1, int y1, int x2, int y2, ll val) {
    upd1(x1, y1, y2, val, -val * (x1 - 1));
    upd1(x2, y1, y2, -val, val * x2);
}

};

```

BIT Update & Query Range :

```

struct BIT {
private:
    int n;
    vector<int> B1, B2;

    void add(vector<int> &b, int idx, int x) {
        ++idx;
    }
}

```

```

    while (idx <= n) {
        b[idx] += x;
        idx += idx & -idx;
    }
}

int sum(vector<int> &b, int idx) {
    idx++;
    int total = 0;
    while (idx > 0) {
        total += b[idx];
        idx -= idx & -idx;
    }
    return total;
}

int prefix(int idx) {
    return sum(B1, idx) * idx - sum(B2, idx);
}

public:

    BIT(int n) : n(n) {
        B1.assign(n + 1, {});
        B2.assign(n + 1, {});
    }

    void update(int l, int r, int x) {
        add(B1, l, x);
        add(B1, r + 1, -x);
        add(B2, l, x * (l - 1));
        add(B2, r + 1, -x * r);
    }

    int query(int i) {
        return prefix(i) - prefix(i - 1);
    }

    int query(int l, int r) {
        return prefix(r) - prefix(l - 1);
    }

};

```

Hilbert Order :

```

inline int64_t hilbertOrder(int x, int y, int pow, int rotate) {
    if (pow == 0) return 0;
    int hpow = 1 << (pow - 1);

```

```

int seg = (x < hpow) ? ((y < hpow) ? 0 : 3) : ((y < hpow) ? 1 : 2);
seg = (seg + rotate) & 3;
static const int rotateDelta[4] = {3, 0, 0, 1};
int nx = x & (x ^ hpow), ny = y & (y ^ hpow);
int nrot = (rotate + rotateDelta[seg]) & 3;
int64_t subSquareSize = int64_t(1) << (2 * pow - 2);
int64_t res = seg * subSquareSize;
int64_t add = hilbertOrder(nx, ny, pow - 1, nrot);
return res + ((seg == 1 || seg == 2) ? add : (subSquareSize - add - 1));
}

struct Query {
    int l, r, idx;
    int64_t ord = -1;

    void calcOrder() {
        ord = hilbertOrder(l, r, 21, 0);
    }

    bool operator<(const Query &other) const {
        return ord < other.ord;
    }
};

int main() {
    int n, q;
    cin >> n >> q;
    for (int i = 0; i < n; ++i)
        cin >> a[i];

    vector<Query> queries(q);
    for (int i = 0; i < q; ++i) {
        cin >> queries[i].l >> queries[i].r;
        queries[i].idx = i;
        queries[i].calcOrder();
    }

    sort(queries.begin(), queries.end());

    int l = 0, r = -1;
    for (auto &qry : queries) {
        while (r < qry.r) add(a[++r]);
        while (r > qry.r) remove(a[r--]);
        while (l < qry.l) remove(a[l++]);
        while (l > qry.l) add(a[--l]);
        ans[qry.idx] = cur_answer;
    }

    for (int i = 0; i < q; ++i)
        cout << ans[i] << '\n';

    return 0;
}

```

SegTreeSD :

```

/// update point / query range
struct SegTree {
    vector<int> seg;
    int N;
    int ign;

    SegTree(int n) {
        N = 1;
        while (N <= n) N <<= 1;
        ign = 0; /// update this
        seg.assign(N << 1, ign);
    }

    int mrg(int a, int b) { return a + b; }

    void update(int i, int val) {
        seg[i += N - 1] = val; /// update this
        while ((i >>= 1) >= 1) {
            seg[i] = mrg(seg[i << 1], seg[i << 1 | 1]);
        }
    }

    int query(int l, int r) {
        int ret = ign, k = 0;
        l += N - 2, r += N - 1;
        while ((l += (1 << k)) <= r) {
            ret = mrg(ret, seg[l >> (k = min(__lg(l & -1), __lg(r - l + 1)))]);
        }
        return ret;
    }
};

```

SegTree2D

```

struct SegTree2d {
    int n, m;
    vector<vector<int>>> seg;

    int neutral() const { return 1e8; }

    int merge(int a, int b) const { return min(a, b); }

    SegTree2d(int n, int m) : n(n), m(m) {

```

```

    seg = vector<vector<int>>(2 * n, vector<int>(2 * m, neutral()));
}

int qry(int x1, int y1, int x2, int y2) const {
    int ret = neutral();
    int y3 = y1 + m, y4 = y2 + m;
    for (x1 += n, x2 += n; x1 <= x2; ++x1 /= 2, --x2 /= 2) {
        for (y1 = y3, y2 = y4; y1 <= y2; ++y1 /= 2, --y2 /= 2) {
            if ((x1 & 1) and (y1 & 1))
                ret = merge(ret, seg[x1][y1]);
            if ((x1 & 1) and !(y2 & 1))
                ret = merge(ret, seg[x1][y2]);
            if (!(x2 & 1) and (y1 & 1))
                ret = merge(ret, seg[x2][y1]);
            if (!(x2 & 1) and !(y2 & 1))
                ret = merge(ret, seg[x2][y2]);
        }
    }
    return ret;
}

void upd(int x, int y, int val) {
    int y2 = y + m;
    for (x += n; x; x /= 2, y = y2) {
        if (x >= n)
            seg[x][y] = val;
        else
            seg[x][y] = merge(seg[2 * x][y], seg[2 * x + 1][y]);
        while (y /= 2)
            seg[x][y] = merge(seg[x][2 * y], seg[x][2 * y + 1]);
    }
}
};

```

Topological :

```

vector<vector<int> > adj;
vector<int> in; /// in-degree of each node
int n; // number of nodes
vector<int> res;

bool Topological() {
    queue<int> q;
    res.clear();
    for (int i = 0; i < n; ++i)
        if (in[i] == 0) q.push(i);
    while (!q.empty()) {
        int u = q.front();
        q.pop();
    }
}

```

```

    res.emplace_back(u);
    for (int v: adj[u])
        if (!--in[v])
            q.push(v);
    }
    return res.size() == n;
}

```

MatPower :

```

typedef vector<ll> row;
typedef vector<row> mat;

mat operator*(const mat &a, const mat &b) {
    int r1 = a.size(), r2 = b.size(), c2 = b[0].size();
    mat ret(r1, row(c2));
    for (int i = 0; i < r1; i++)
        for (int j = 0; j < c2; j++)
            for (int k = 0; k < r2; k++)
                ret[i][j] += a[i][k] * b[k][j], ret[i][j] %= mod;
    return ret;
}

mat operator^(const mat &a, ll k) {
    mat ret(a.size(), row(a.size()));
    for (int i = 0; i < a.size(); i++) ret[i][i] = 1;
    for (mat tmp = a; k; tmp = tmp * tmp, k /= 2) if (k & 1) ret = ret * tmp;
    return ret;
}

```

XOR Basis :

```

template<typename T = int, int B = 31>
struct Basis {
    T a[B];
    Basis() {
        memset(a, 0, sizeof a);
    }
    void insert(T x){
        for (int i = B - 1; i >= 0; i--) {
            if (x >> i & 1) {
                if (a[i] ^ x) x ^= a[i];
                else {
                    a[i] = x;
                    break;
                }
            }
        }
    }
};

```

```

    }
    }
}
bool can(T x) {
    for(int i = B - 1; i >= 0; i--) {
        x = min(x, x ^ a[i]);
    }
    return x == 0;
}
T max_xor(T ans = 0) {
    for(int i = B - 1; i >= 0; i--) {
        ans = max(ans, ans ^ a[i]);
    }
    return ans;
}
};

```

CRT :

```

#define ll long long
#define M first
#define R second
typedef pair<ll, ll> mod_eq;

void nxt_r(ll &r0, ll &r1, ll &r) {
    int r2 = r0 - r * r1;
    r0 = r1, r1 = r2;
}

// r0 = a , r1 = b
// return gcd = a * x0 + b * y0
ll egcd(ll r0, ll r1, ll &x0, ll &y0) {
    ll x1 = y0 = 0, y1 = x0 = 1;
    while (r1) {
        ll r = r0 / r1;
        nxt_r(r0, r1, r);
        nxt_r(x0, x1, r);
        nxt_r(y0, y1, r);
    }
    return r0;
}

// c = x * a + y * b
// x` = x - (b / g) * k , y` = y + (a / g) * k - for any k
bool solveLDE(ll a, ll b, ll c, ll &x, ll &y, ll &g) {
    g = egcd(a, b, x, y);
    ll m = c / g;
    x *= m, y *= m;
}

```

```

    return m * g == c;
}

bool mod_inv(ll a, ll mod, ll &x) {
    ll y, g = egcd(a, mod, x, y);
    if (g != 1) {
        return false; // a and mod are not co-prime
    }
    x %= mod;
    if (x < 0) x += mod;
    return true;
}

bool CRT(const mod_eq &e1, const mod_eq &e2, mod_eq &res) {
    ll q1, q2, g;
    if (!solveLDE(e1.M, -e2.M, e2.R - e1.R, q1, q2, g)) {
        return false;
    }
    q2 %= e1.M / g;
    ll lcm = abs(e1.M / g * e2.M);
    //// ll x = ((__int128) e2.M * (__int128) q2 + (__int128) e2.R) % lcm; ??
    ll x = e2.M * q2 + e2.R;
    x %= lcm;
    if (x < 0) x += lcm;
    res = {lcm, x};
    return true;
}

bool CRT(const vector<mod_eq> &eq, mod_eq &ret) {
    ret = eq[0];
    for (int i = 1; i < eq.size(); ++i) {
        if (!CRT(eq[i], ret, ret)) {
            return false;
        }
    }
    return true;
}

```

Gauss

```

int Gauss(vector<vector<int>> a, vector<int> &ans) {
    int n = a.size(), m = (int) a[0].size() - 1;
    vector<int> pos(m, -1);
    int free_var = 0;
    const ll MODSQ = (ll) mod * mod;
    int det = 1, rank = 0;
    for (int col = 0, row = 0; col < m && row < n; col++) {
        int mx = row;
        for (int k = row; k < n; k++) if (a[k][col] > a[mx][col]) mx = k;
    }
}

```

```

if (a[mx][col] == 0) {
    det = 0;
    continue;
}
for (int j = col; j <= m; j++) swap(a[mx][j], a[row][j]);
if (row != mx) det = det == 0 ? 0 : mod - det;
det = 1LL * det * a[row][col] % mod;
pos[col] = row;
int inv = power(a[row][col], mod - 2);
for (int i = 0; i < n && inv; i++) {
    if (i != row && a[i][col]) {
        int x = ((1LL) a[i][col] * inv) % mod;
        for (int j = col; j <= m && x; j++) {
            if (a[row][j]) a[i][j] = (MODSQ + a[i][j] - ((1LL) a[row][j] * x)) % mod;
        }
    }
}
row++;
++rank;
}
ans.assign(m, 0);
for (int i = 0; i < m; i++) {
    if (pos[i] == -1) free_var++;
    else ans[i] = ((1LL) a[pos[i]][m] * power(a[pos[i]][i], mod - 2)) % mod;
}
for (int i = 0; i < n; i++) {
    1LL val = 0;
    for (int j = 0; j < m; j++) val = (val + ((1LL) ans[j] * a[i][j])) % mod;
    if (val != a[i][m]) return -1; //no solution
}
return free_var; //has solution
}

```

Gauss - doubles

```

const double eps = 1e-9;

int Gauss(vector<vector<double>> a, vector<double> &ans) {
    int n = (int) a.size(), m = (int) a[0].size() - 1;
    vector<int> pos(m, -1);
    double det = 1;
    int rank = 0;
    for (int col = 0, row = 0; col < m && row < n; ++col) {
        int mx = row;
        for (int i = row; i < n; i++) if (fabs(a[i][col]) > fabs(a[mx][col])) mx = i;
        if (fabs(a[mx][col]) < eps) {
            det = 0;
            continue;
        }
    }
}

```

```

for (int i = col; i <= m; i++) swap(a[row][i], a[mx][i]);
if (row != mx) det = -det;
det *= a[row][col];
pos[col] = row;
for (int i = 0; i < n; i++) {
    if (i != row && fabs(a[i][col]) > eps) {
        double c = a[i][col] / a[row][col];
        for (int j = col; j <= m; j++) a[i][j] -= a[row][j] * c;
    }
}
++row, ++rank;
}
ans.assign(m, 0);
for (int i = 0; i < m; i++) {
    if (pos[i] != -1) ans[i] = a[pos[i]][m] / a[pos[i]][i];
}
for (int i = 0; i < n; i++) {
    double sum = 0;
    for (int j = 0; j < m; j++) sum += ans[j] * a[i][j];
    if (fabs(sum - a[i][m]) > eps) return -1; //no solution
}
for (int i = 0; i < m; i++) if (pos[i] == -1) return 2; //infinte solutions
return 1; //unique solution
}

```

Tree Hashing :

```

#define ull unsigned long long
#define all(X) X.begin(),X.end()

ull pw(ull b, ull p) {
    if (!p) return 1;
    ull ret = pw(b, p >> 1);
    ret *= ret;
    if (p & 1)
        ret = ret * b;
    return ret;
}

ull dfs(int u, int par) {
    vector<ull> child;
    for (auto v: adj[u]) {
        if (v == par) continue;
        child.push_back(dfs(v, u));
    }
    sort(all(child));
    ull ret = 0;
    for (int i = 0; i < child.size(); ++i) {
        ret += child[i] * child[i] + child[i] * pw(31, i + 1) + (ull) 42;
    }
}

```

```

    }
    return ret;
}

```

LIS DP deff :

```

int lengthOfLIS(ll nums[],int n)
{
    vector<ll> ans;
    ans.push_back(nums[0]);
    for (int i = 1; i < n; i++) {
        if (nums[i] > ans.back()) {
            ans.push_back(nums[i]);
        }
        else {
            int low = lower_bound(ans.begin(), ans.end(),nums[i])- ans.begin();
            ans[low] = nums[i];
        }
    }
    return ans.size();
}

```

LCS Seg Tree Dp :

```

#include<bits/stdc++.h>
using namespace std;
#define ll long long
struct FenwickTree{
    int N;vector<int>bit;
    FenwickTree(int n,int init=0)
    {
        N=n+1;
        bit=vector<int>(n+1,init);
    }
    void update(int idx,int val)
    {
        while(idx<N)
        {
            bit[idx]=max(bit[idx],val);
            idx+=idx&-idx;
        }
    }
    int query(int idx)
    {

```

```

        int ret=0;
        while(idx>0)
        {
            ret=max(ret,bit[idx]);
            idx-=idx&-idx;
        }
        return ret;
    }
};
signed main()
{
    ios_base::sync_with_stdio(0);cin.tie(0);cout.tie(0);
    int n;cin>>n;
    int pos[n+1];
    for(int i=1;i<=n;i++)
    {
        int x;cin>>x;
        pos[x]=i;
    }
    pair<int,int>s[n];
    for(int i=0;i<n;i++)
    {
        cin>>s[i].first;
        s[i].second=i+1;
    }
    sort(s,s+n,[&](pair<int,int>&i,pair<int,int>&j){
        return pos[i.first]<pos[j.first];
    });
    FenwickTree tree(n);
    for(int i=0;i<n;i++)
    {
        int pre=tree.query(s[i].second);
        tree.update(s[i].second,pre+1);
    }
    cout<<tree.query(n);
    return 0;
}

```

coordinateCoprression :

```

struct coordinateCoprression{
private:
    vector<ll>init;
    void compress(vector<ll>&v)
    {
        sort(v.begin(),v.end());
        v.erase(unique(v.begin(),v.end()),v.end());
    }
public:

```

```

coordinateCoprime(vector<ll>&v)
{
    init=v;
    compress(init);
}
int index(ll val)
{
    return lower_bound(init.begin(),init.end(),val)-init.begin();
}
ll initVal(int idx)
{
    return init[idx];
}
};

```

Solve equation :

```

pair<ll,ll> equ(ll a,ll b,ll c)
{
    ll x1 = (-b + sqrt(b * b - 4 * a * c)) / (2 * a);
    ll x2 = (-b - sqrt(b * b - 4 * a * c)) / (2 * a);
    return {x1,x2};
}

```

Bit Mask :

```

#include<bits/stdc++.h>
using namespace std;
#define ll long long
bool Knowbit(ll n,int i)
{
    return (n>>i)&1;
}
ll Setbit(ll n,int i)
{
    return n|(1<<i);
}
ll Resetbit(ll n,int i)
{
    return n&(~(1<<i));
}
ll flip(ll n,int i)
{
    return n^(1<<i);
}

```

```

}
bool isPowerOfTwo(int n)
{
    if(n==0)return 0;
    return !(n&(n-1));
}
signed main()
{
    ios_base::sync_with_stdio(0);cin.tie(0);cout.tie(0);
    int n;cin>>n;
    int s[n];
    for(int i=0;i<n;i++) cin>>s[i];
    int ans=0;
    for(int mask=0;mask<(1<<n);mask++)
    {
        int even=0,odd=0;
        for(int i=0;i<n;i++)
        {
            if(Knowbit(mask,i)==1)
            {
                if(s[i]&1) odd++;
                else even++;
            }
        }
        if(even>odd) ans++;
    }
    cout<<ans;
    return 0;
}

```

Maximum Bipartite Matching :

```

#include<bits/stdc++.h>
using namespace std;
#define ll long long
const int N=105;
int mat[N];vector<int>adj[N];bool vis[N];
bool can_match(int node)
{
    if(vis[node]) return 0;
    vis[node]=1;
    for(auto it:adj[node])
    {
        if(mat[it]==-1||can_match(mat[it]))
        {
            mat[it]=node;
            return 1;
        }
    }
}

```

```

    return 0;
}
signed main()
{
    ios_base::sync_with_stdio(0);cin.tie(0);cout.tie(0);
    int n,m,q;cin>>n>>m>>q;
    set<int>st;map<string,int>idx;int c=1;
    for(int i=0;i<q;i++)
    {
        string a,b;cin>>a>>b;
        if(idx[a]==0) idx[a]=c++;
        if(idx[b]==0) idx[b]=c++;
        adj[idx[a]].push_back(idx[b]);
        st.insert(idx[a]);
    }
    memset(mat,-1,sizeof mat);
    int ans=0;
    for(auto it:st) memset(vis,0,sizeof vis),ans+=can_match(it);
    cout<<ans;
    return 0;
}
// https://codeforces.com/gym/103800/problem/J

```

Persistent Binary Trie:

```

#define ull unsigned long long
ull pw1[50], pw2[50];

struct node {
    node *l, *r;
    ull h;

    node() : l(nullptr), r(nullptr), h(0) {}

    node(node *l, node *r, int lvl) {
        this->l = this->r = nullptr, h = 0;
        if (l) this->l = l, h = pw1[lvl] * l->h;
        if (r) this->r = r, h += pw2[lvl] * r->h;
    }

    node(ull val) {
        l = r = nullptr;
        h = ((ull) (1e9 + 345) ^ val) * (3 * val + 654);
    }
};

node *update(int val, node *x, int lvl) {
    if (!lvl) {
        if (x)return nullptr;
    }
}

```

```

    return new node(val);
}
if (val & (1 << lvl)) {
    return new node(
        x ? x->l : nullptr,
        update(val, x ? x->r : nullptr, lvl - 1),
        lvl + 1
    );
}
return new node(
    update(val, x ? x->l : nullptr, lvl - 1),
    x ? x->r : nullptr,
    lvl + 1
);
}

int query(node *l, node *r, int msk, int lvl) {
    if ((l ? l->h : 0) == (r ? r->h : 0)) return -1;
    if (!~lvl) return msk;
    int lf = query(l ? l->l : l, r ? r->l : r, msk, lvl - 1);
    if (~lf) return lf;
    return query(l ? l->r : l, r ? r->r : r, msk | (1 << lvl), lvl - 1);
}

void pre() {
    pw1[0] = pw2[0] = 1;
    for (int i = 1; i < 32; ++i) {
        pw1[i] = pw1[i - 1] * 31;
        pw2[i] = pw2[i - 1] * 53;
    }
}

```

Persistent Dsu:

```

struct DSU {
    vector<vector<pair<int, int>>> par;
    int time = 0; // initial time
    DSU(int n) : par(n + 1, {{-1, 0}}) {}

    bool merge(int u, int v) {
        ++time;
        if ((u = root(u, time)) == (v = root(v, time))) return 0;
        if (par[u].back().first > par[v].back().first)
            swap(u, v);
        par[u].emplace_back(par[v].back().first + par[v].back().first, time);
        par[v].emplace_back(u, time); //par[v] = u
        return 1;
    }
}

```

```
bool same(int u, int v, int t) {
    return root(u, t) == root(v, t);
}

int root(int u, int t) {
    if (par[u].back().first >= 0 && par[u].back().second <= t)
        return root(par[u].back().first, t);
    return u;
}

int size(int u, int t) { //size of the component of u at time t
    u = root(u, t);
    int l = 0, r = (int) par[u].size() - 1, ans = 0;
    while (l <= r) {
        int mid = l + r >> 1;
        if (par[u][mid].second <= t) ans = mid, l = mid + 1;
        else r = mid - 1;
    }
    return -par[u][ans].first;
}

int first_time_to_be_same(int u, int v) {
    int ans = -1, l = 0, r = time;
    while (l <= r) {
        int mid = l + r >> 1;
        if (same(u, v, mid)) ans = mid, r = mid - 1;
        else l = mid + 1;
    }
    return ans;
}
};
```